

ySpaceOS Mission Control

Autonomous Logistics Control Tower

Product Requirements Document — Master Specification

Product: ySpaceOS

Module: Mission Control

Product Type: Autonomous Logistics Control Tower

Primary Users: ySpace Operations, Fleet Operations, Mission Operators, Logistics Managers, Autonomy Engineers, Maintenance Teams, Administrators

Initial Focus: Drone logistics operations

Version: V1 + V2 Product Roadmap

Document Purpose: Product, UX/UI, engineering, autonomy, and operational specification

1. EXECUTIVE SUMMARY

ySpaceOS Mission Control is ySpace's autonomous logistics control tower—a unified operational interface that transforms real-time fleet data, spatial intelligence, mission information, telemetry, AI perception, route intelligence, and logistics data into a single system through which operators can monitor, plan, control, and progressively automate the movement of goods through the physical world.

ySpace is building a logistics and transportation company whose long-term advantage is not simply owning vehicles or drones. ySpaceOS is the operating system. ySpace Mission Control is the logistics control-tower application/interface that sits on top of ySpaceOS.

The advantage is **intelligence**.

The company needs a system capable of understanding:

- where its assets are,
- what they are doing,
- what they are carrying,
- where they are going,
- what exists around them,
- what could go wrong,
- what route they should take,
- what mission they should perform,
- when they should return,
- when they should recharge,

- and eventually, what the system should do without a human having to make every decision.

That system is **ySpaceOS**.

At the operational center of ySpaceOS is **Mission Control**.

Mission Control is the **Autonomous Logistics Control Tower**.

It is the primary interface through which ySpace's human operations team interacts with its physical transportation infrastructure.

It should feel less like a conventional fleet-management dashboard and more like a **modern command center for an autonomous transportation network**.

The system combines:

Command & Control + Mission Planning + Route Intelligence + Spatial Intelligence + Telemetry + Fleet Intelligence + Delivery Operations + AI Perception + Autonomous Decision Support

into one operational environment.

The first version should not attempt to automate everything.

Instead, Version 1 establishes the foundation:

1. Command & Control
2. Fleet Monitoring
3. Mission Management
4. Autonomous Mission Planning
5. Dynamic Path Planning
6. Autonomous Route Intelligence
7. Obstacle Awareness / Avoidance Integration
8. Telemetry
9. Spatial Intelligence
10. AI Detection & Tracking
11. Active Delivery Operations
12. Airspace / Geofence Intelligence
13. Alerts & Incident Management
14. Bases & Charging Visibility
15. Mission History
16. Operational Analytics

Version 2 then expands this foundation toward increasingly predictive and autonomous operations:

1. Predictive Fleet Health
2. Autonomous Charging Intelligence
3. Logistics Risk Engine
4. Predictive Mission Intelligence
5. Predictive / Autonomous Tagging Intelligence
6. Advanced anomaly detection
7. Multi-drone coordination
8. Automated mission recovery
9. More autonomous operational decisions
10. Advanced logistics forecasting
11. Digital Air Corridor optimization
12. Higher-level fleet autonomy

The philosophy is simple:

Version 1 helps humans operate an intelligent fleet.

Version 2 increasingly allows the system to operate the fleet intelligently with humans supervising it.

2. THE ySpaceOS CONCEPT

2.1 What is ySpaceOS?

ySpaceOS is the operating system for ySpace's autonomous logistics infrastructure.

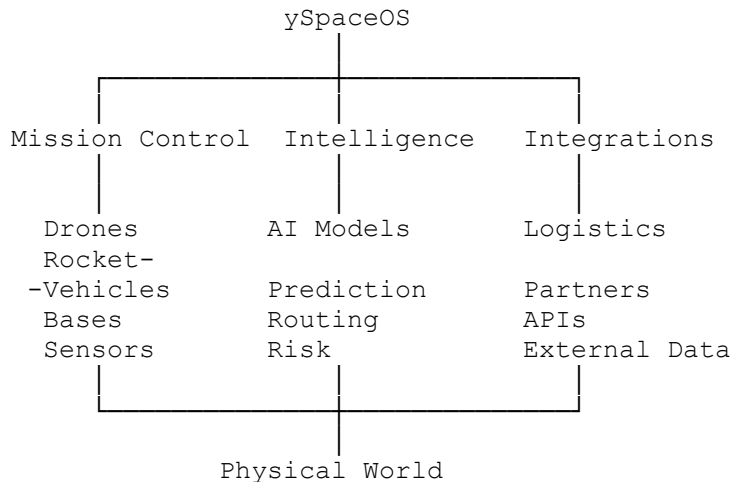
The term "operating system" should not be interpreted as a conventional desktop or mobile operating system.

It is the software intelligence layer that connects:

- physical transportation assets,
- sensors,
- rocket vehicles,
- drones,
- missions,
- maps,
- logistics jobs,
- telemetry,
- AI models,
- autonomy systems,
- operators,
- customers,

- infrastructure,
- and operational data.

The architecture should eventually resemble:



3. WHAT IS ySpace MISSION CONTROL?

Mission Control is the **human-facing logistics control tower of ySpaceOS**.

It is not the operating system itself.

Rather:

ySpaceOS = underlying operational intelligence and infrastructure

Mission Control = control-tower interface for operating that infrastructure

This distinction is extremely important for the product architecture.

The designer should therefore avoid creating a product that feels like:

"Drone Tracking Software."

Instead, the interface should communicate:

"This is where ySpace operates its autonomous transportation network."

4. THE CORE PRODUCT PROMISE

Mission Control should allow an authorized operator to answer five questions immediately.

1. What is happening?

What is the current state of the network?

2. Where is it happening?

Where are the drones, deliveries, incidents, bases, routes, obstacles, and operational zones?

3. Why is it happening?

What caused the current situation?

4. What will happen next?

What does the system predict?

5. What should we do?

What action does the system recommend or automatically execute?

That is the central UX philosophy.

5. DESIGN PRINCIPLE: SITUATIONAL AWARENESS

Mission Control should prioritize **situational awareness** over simply displaying data.

A control tower operator should be able to glance at the screen and understand the operation.

For example:

```
NETWORK
23 ACTIVE DRONES
18 ACTIVE MISSIONS
2 CHARGING
1 ATTENTION REQUIRED
0 CRITICAL
```

AIRSPACE
CLEAR

DELIVERIES
17 ON TIME
1 DELAYED

AUTONOMY
91% NOMINAL

SYSTEM
OPERATIONAL

The interface should not require the operator to open six different pages to understand this.

6. DESIGN PHILOSOPHY

The interface should follow six principles.

6.1 Map-first

The physical world is central to logistics.

Therefore, the map should be one of the primary elements of the interface.

6.2 Information hierarchy

Not every piece of information deserves equal visual weight.

Critical information must dominate.

Normal telemetry should remain available without overwhelming the operator.

6.3 Progressive disclosure

Show the most important information first.

Allow the operator to drill deeper when necessary.

6.4 Human-in-the-loop

Version 1 should not pretend the system is completely autonomous.

The operator must remain able to:

- approve,
- modify,
- pause,
- reroute,
- cancel,
- intervene,
- and override.

6.5 Automation transparency

Whenever AI recommends something, the operator should understand:

what happened → why → recommended action → confidence → consequence

6.6 Exception-first operations

The system should not force operators to stare at every drone continuously.

Instead:

Normal operations disappear into the background. Exceptions demand attention.

7. INDUSTRY DESIGN BENCHMARK

The architecture should learn from the direction of modern drone-management and autonomous-system platforms without copying them.

The FAA describes UTM as a framework involving flight planning, authorization, surveillance and conflict management, particularly as BVLOS operations scale. ([Federal Aviation Administration](#))

EASA's U-space framework similarly identifies network identification, geo-awareness, flight authorization and traffic information as core services for scalable drone operations. ([EASA](#))

NASA's UTM research demonstrated concepts involving flight-plan submission, conflict checking, geofencing, scheduling and multi-drone tracking. ([NASA](#))

Commercial platforms are also moving beyond simple fleet tracking. DJI's FlightHub 2, for example, includes mission planning, virtual cockpit functionality, AI object detection, cloud mapping and route-management capabilities. ([DJI Official](#))

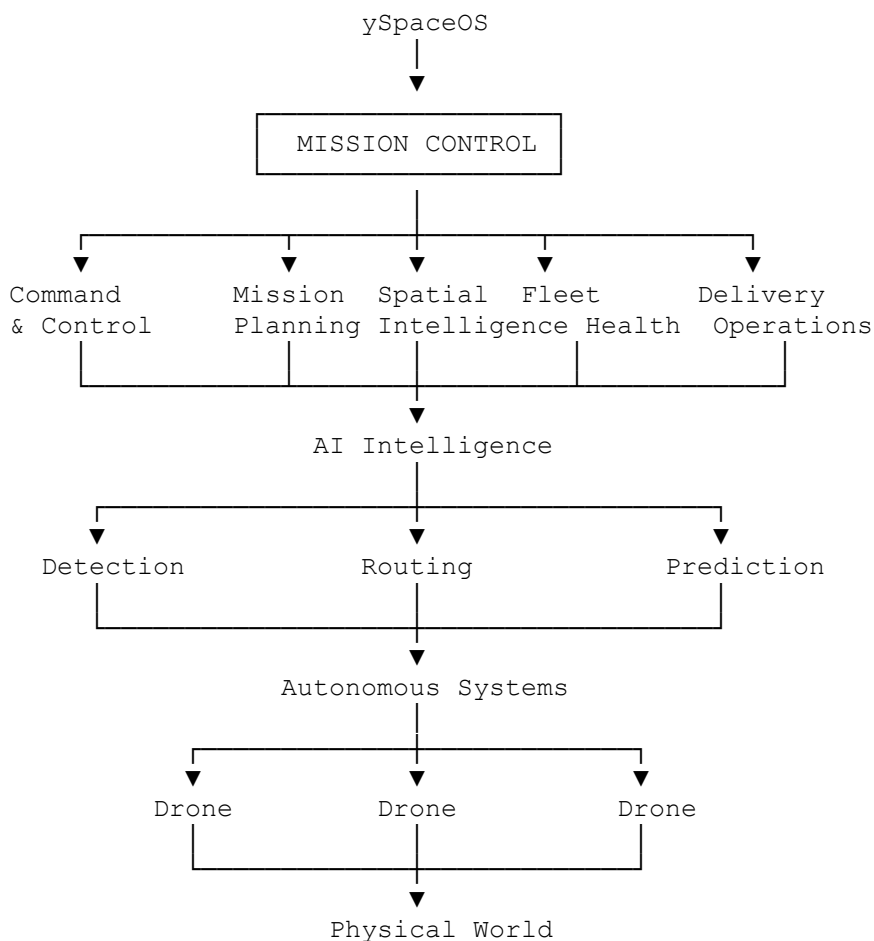
At the autonomy-software level, NVIDIA's Isaac ROS provides modular robotics perception and navigation capabilities designed to run across workstations and embedded systems such as Jetson. ([NVIDIA Developer](#))

Anduril's Lattice demonstrates another important architectural reference: command and control, mission autonomy, sensor integration and autonomous systems can be combined within a broader software platform rather than treated as disconnected tools. ([Anduril](#))

ySpace Mission Control should therefore be designed from day one as an **extensible operational platform**, not as a static dashboard.

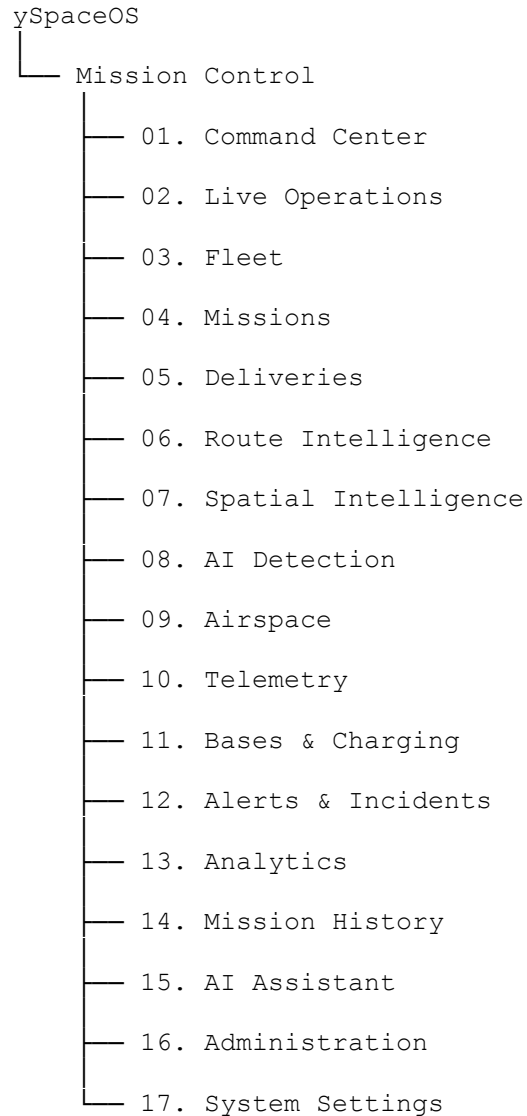
8. PRODUCT ARCHITECTURE

At a high level:



9. COMPLETE FEATURE HIERARCHY

The Mission Control hierarchy should be:



Version 2 expands these modules rather than creating an entirely different product.

10. VERSION 1 — PRODUCT OBJECTIVE

Version 1 should answer:

Can ySpace operate a real autonomous drone logistics fleet from one intelligent command center?

The answer should be yes.

Version 1 therefore focuses on **operational visibility, planning, control, intelligence and supervised autonomy**.

11. VERSION 1 — CORE MODULES

V1.1 Command & Control

This is the foundation.

Operators must be able to:

- view all drones,
- select a drone,
- inspect status,
- send commands,
- start a mission,
- pause a mission,
- resume a mission,
- cancel a mission,
- return to base,
- reroute,
- land,
- change mission parameters,
- acknowledge alerts,
- and escalate incidents.

Critical command hierarchy

NORMAL

↓

MONITOR

ATTENTION

↓

INVESTIGATE

WARNING

↓

INTERVENE / AUTHORIZE

CRITICAL
↓
EMERGENCY ACTION

12. COMMAND CENTER SCREEN

The main Mission Control screen should be the most important UI screen.

It should contain:

Header

- ySpaceOS logo
- Mission Control label
- operational status
- current time
- network connectivity
- operator identity
- notification center
- system status

Left navigation

- Overview
- Live Operations
- Fleet
- Missions
- Deliveries
- Routes
- Spatial Intelligence
- AI Detection
- Airspace
- Telemetry
- Bases
- Alerts
- Analytics
- History
- Settings

Center

Large interactive 2D/3D map.

Right panel

Contextual operational information.

Bottom

Mission timeline / activity stream.

13. THE MAIN MAP

The map is arguably the most important visual component of Mission Control.

The primary map should use **satellite imagery as the default operational view**, providing the geographic foundation for the entire logistics control tower.

It should support:

- 3D map
- Satellite imagery
- Roads
- Terrain
- Drone locations
- Routes
- Corridors
- Landing zones
- Bases
- Charging stations
- Geofences
- Restricted zones
- Weather
- Wind
- Detected objects
- Delivery locations
- Incidents

The map should not simply be decorative.

It is an **operational interface**.

13.1 The 3D Map Should Be a Separate Interactive Layer

Rather than attempting to replace or modify the existing satellite map with a fully detailed 3D representation, Yspace should introduce a **separate, clickable 3D Map View**.

The satellite map remains the primary global map because it can immediately provide geographic coverage across large areas.

A high-quality 3D environment, however, is fundamentally different.

To create accurate 3D streets, buildings, rooftops, infrastructure and other environmental details, Yspace will likely need to collect and process its own **camera data, aerial imagery, spatial data and other mapping information**.

This means the 3D environment will not necessarily be created for an entire country or continent at once.

Instead, Yspace can progressively build its 3D world **one city, district, or street at a time**.

For example:

Satellite Map → Select Location → Enter 3D View

A user could be viewing Lagos through the satellite map and click a **3D View** button. If detailed 3D mapping is available for that specific area, Mission Control transitions into the Yspace 3D environment.

Over time, more locations become available.

This creates two complementary mapping systems:

Satellite Map

- Global geographic coverage
- Primary operational map
- Routes and logistics monitoring
- Real-time fleet visibility
- Weather and environmental data
- Geofences and restricted areas

Yspace 3D Map

- High-detail environments
- Buildings and rooftops
- Streets and infrastructure
- Drone-level visualization
- Landing-zone visualization
- Detailed mission planning
- Areas mapped and built by Yspace

The objective is therefore **not to improve or replace the existing satellite map's underlying 3D capabilities**.

Instead, Yspace builds its own **proprietary 3D mapping layer on top of the geographic foundation**, gradually expanding coverage as more camera and spatial data is collected.

This also creates a long-term strategic asset for Yspace.

Every newly mapped street, building, rooftop, landing zone and logistics environment can become part of Yspace's proprietary **3D logistics map**, making the platform increasingly valuable as its geographic coverage expands.

14. MAP INTERACTION

Operators should be able to:

- zoom,
- rotate,
- tilt,
- pan,
- select drones,
- select routes,
- inspect locations,
- measure distances,
- inspect altitude,
- create waypoints,
- modify routes,
- create operational zones,
- inspect detected objects,
- inspect airspace restrictions,
- inspect historical routes.

15. DRONE REPRESENTATION

Each drone should have a visual representation.

The operator should immediately distinguish:

- flying,
- idle,
- charging,
- preparing,
- mission complete,

- returning,
- warning,
- emergency,
- offline.

Each drone marker should be interactive.

Clicking a drone opens a contextual card.

Example:

YS-DRONE-014

STATUS
IN MISSION

MISSION
DELIVERY #D-10492

BATTERY
67%

ALTITUDE
84m

SPEED
14.2m/s

ETA
08:42

ROUTE
AUTONOMOUS

HEALTH
NORMAL

16. DRONE DETAIL PAGE

The drone detail screen should provide a complete digital profile.

Identity

- Drone ID
- Model
- Serial number
- Software version
- Firmware
- Assignment

Current State

- Location
- Altitude
- Speed
- Heading
- Battery
- Flight mode
- Connectivity
- GPS status

Mission

- Current mission
- Origin
- Destination
- Progress
- ETA
- Route
- Mission status

Health

- Battery health
- Motor status
- Sensor status
- Temperature
- Communications
- Navigation health

History

- Previous missions
- Flight hours
- Distance traveled
- Incidents
- Maintenance

17. MISSION MANAGEMENT

A mission is the fundamental operational unit.

A mission should contain:

Mission ID
Mission Type
Payload
Origin
Destination
Priority
Assigned Drone
Route
Start Time
Expected Completion
Actual Completion
Mission Status
Operator
Autonomy Level
Exceptions
Telemetry
Outcome

Mission statuses:

- Draft
- Planned
- Awaiting Approval
- Approved
- Preparing
- Active
- Paused
- Rerouting
- Returning
- Completed
- Failed
- Cancelled

18. MISSION CREATION

The operator should be able to create a mission manually.

Step 1

Select origin.

Step 2

Select destination.

Step 3

Enter package information.

Step 4

Set priority.

Step 5

Select or allow ySpaceOS to recommend a drone.

Step 6

Generate route.

Step 7

Review route.

Step 8

Run mission validation.

Step 9

Approve.

Step 10

Launch.

19. AUTONOMOUS MISSION PLANNING

This is one of the most important Version 1 features.

The operator should not have to manually create every waypoint.

The system should be capable of generating a proposed mission plan based on:

- origin,
- destination,
- drone capabilities,
- battery,

- payload,
- airspace,
- geofences,
- terrain,
- buildings,
- known obstacles,
- weather,
- wind,
- operational restrictions,
- and mission priority.

The system returns:

Recommended Mission Plan

with:

- route,
- estimated duration,
- estimated energy consumption,
- risk level,
- alternative routes,
- predicted arrival.

The operator can then approve or modify it.

20. AUTONOMOUS ROUTE INTELLIGENCE

This must be a **Version 1 feature**.

It is one of the features that makes ySpace Mission Control significantly more ambitious than conventional fleet-management software.

The system should understand the environment through spatial data and calculate routes accordingly.

The route engine should consider:

Static information

- buildings,

- terrain,
- towers,
- bridges,
- power infrastructure,
- restricted zones,
- designated corridors,
- landing zones.

Dynamic information

- weather,
- wind,
- temporary restrictions,
- detected obstacles,
- other aircraft,
- network conditions,
- congestion,
- operational events.

Vehicle information

- battery,
- payload,
- capabilities,
- maximum range,
- maximum altitude,
- current health.

21. DYNAMIC PATH PLANNING

The route is not fixed forever.

If conditions change, the route engine should be able to generate an alternative.

For example:

ORIGINAL ROUTE

BASE

↓

CORRIDOR A

↓

DESTINATION

Wind changes:

NEW CONDITION
↓
ROUTE RISK INCREASED
↓
ALTERNATIVE GENERATED
↓
CORRIDOR B
↓
DESTINATION

The operator sees:

Route updated because wind conditions increased risk along Corridor A.

22. DIGITAL AIR CORRIDORS

Digital Air Corridors should be implemented within Mission Control in Version 1.

A corridor is a digitally defined operational route through airspace.

The system can represent:

- corridor boundaries,
- altitude ranges,
- direction,
- operational restrictions,
- permitted vehicle classes,
- risk level,
- traffic,
- weather,
- capacity,
- and historical performance.

The concept aligns with the broader direction of digital airspace management, where geospatial constraints, flight planning and traffic information become machine-readable operational inputs. ([Federal Aviation Administration](#))

23. OBSTACLE AVOIDANCE

There are two distinct concepts that must not be confused.

Mission Control

Mission Control plans and supervises the route.

Onboard autonomy

The drone's onboard system performs real-time perception and immediate collision avoidance.

Mission Control should therefore receive:

- obstacle detections,
- collision warnings,
- avoidance events,
- emergency reroutes,
- perception status.

The PRD should **not** assume that the cloud dashboard itself can perform millisecond-level collision avoidance.

That belongs closer to the aircraft.

This distinction is critical for engineering architecture.

24. SPATIAL INTELLIGENCE

Spatial Intelligence is the system's understanding of the physical environment.

It should combine:

- maps,
- satellite imagery,
- 3D terrain,
- building data,
- sensor data,
- drone imagery,
- LiDAR where available,
- GPS,
- weather,
- airspace information,
- detected objects.

The goal is:

Turn geographic data into operational understanding.

Modern perception systems demonstrate the value of combining 3D sensing with object detection, classification, tracking and spatial visualization. Ouster, for example, describes systems that detect, classify and track objects while producing geo-position, speed and direction information. ([Ouster Documentation](#))

25. AI DETECTION & TRACKING

This is also **Version 1**.

The system should be able to ingest perception data and represent detected objects.

Potential object classes:

- person,
- vehicle,
- building,
- drone,
- aircraft,
- bird,
- obstacle,
- infrastructure,
- landing area,
- unknown object.

The system should track:

- object ID,
- class,
- location,
- velocity,
- direction,
- confidence,
- first detected,
- last detected.

26. AI DETECTION SCREEN

The designer should create a dedicated screen where operators can see:

AI DETECTION

ACTIVE OBJECTS: 37

DRONES	4
VEHICLES	18
PEOPLE	9
OBSTACLES	5
UNKNOWN	1

Clicking an object reveals:

OBJECT #9281

CLASS
Vehicle

CONFIDENCE
96%

POSITION
6.43xxxxx, 3.42xxxxx

SPEED
31 km/h

DIRECTION
NE

FIRST DETECTED
14:31:08

LAST UPDATE
14:31:13

27. AI TRACKING

The system should maintain object tracks rather than merely detecting individual frames.

Conceptually:

Frame 1 → Object detected
Frame 2 → Same object
Frame 3 → Same object
Frame 4 → Object moved
Frame 5 → Object moved

The system therefore understands:

This is the same object moving through space.

This becomes increasingly valuable for:

- dynamic obstacles,
- vehicles,
- other drones,
- landing zones,
- security,
- route planning.

28. TELEMETRY

Telemetry is the nervous system of Mission Control.

The system should receive and visualize:

Flight

- GPS
- altitude
- speed
- heading
- climb rate
- flight mode

Energy

- battery percentage
- voltage
- current
- energy consumption
- estimated remaining flight time

Vehicle

- motor state
- temperature
- vibration
- sensor state
- controller state

Communication

- signal strength

- latency
- connection status
- data-link quality

Mission

- waypoint
- route progress
- ETA
- mission state

29. TELEMETRY VISUALIZATION

Do not display everything simultaneously.

Use:

Level 1

Operational summary.

Level 2

Current telemetry.

Level 3

Historical telemetry.

Level 4

Engineering telemetry.

The average operations manager should not see engineering-level sensor data unless needed.

30. LIVE DELIVERY OPERATIONS

Mission Control should also display the logistics layer.

Every delivery should have:

- Delivery ID
- Package ID
- Customer
- Pickup
- Destination
- Vehicle/drone
- Status
- ETA
- Route
- Priority
- Exceptions.

Example:

DELIVERY #10492

STATUS
IN TRANSIT

ASSET
YS-DRONE-014

ORIGIN
YSPACE HUB A

DESTINATION
VICTORIA ISLAND

ETA
08:42

DISTANCE REMAINING
3.4 KM

STATUS
ON TIME

31. DELIVERY BOARD

The UI should provide a board/table view.

DELIVERY	ASSET	STATUS	ETA
#10492	D-014	IN TRANSIT	08:42
#10493	D-009	PREPARING	08:51
#10494	D-017	DELAYED	09:12
#10495	D-011	COMPLETED	—

Operators should be able to filter by:

- status,
- location,
- asset,
- priority,
- delay,
- customer,
- route.

32. AIRSPACE INTELLIGENCE

Mission Control should maintain an operational airspace layer.

This includes:

- restricted zones,
- geofences,
- approved corridors,
- temporary restrictions,
- nearby aircraft information where available,
- operational boundaries,
- launch/landing areas.

The broader regulatory ecosystem increasingly treats identification, geo-awareness, flight authorization and traffic information as foundational services for scalable drone operations.

[\(EASA\)](#)

33. GEOFENCING

The system should support:

Hard geofence

Drone must not enter.

Soft geofence

Drone may enter only under specific conditions.

Operational zone

Normal operating area.

Restricted zone

No autonomous operation.

Emergency zone

Special rules.

The map should visually communicate these zones.

34. WEATHER INTELLIGENCE

Version 1 should integrate weather data into route planning.

The system should display:

- wind,
- wind direction,
- precipitation,
- visibility,
- temperature,
- storms,
- relevant alerts.

Weather should influence route recommendations.

Example:

Wind conditions increasing energy consumption by estimated 12%.

The operator can then review the alternative route.

35. BASE MANAGEMENT

Mission Control should display all operational bases.

Each base should include:

- location,
- available drones,
- charging capacity,
- active missions,
- available charging slots,
- operational status.

Example:

YSPACE BASE 01

DRONES
12

AVAILABLE
5

IN MISSION
6

CHARGING
1

CHARGERS
4 / 6 OCCUPIED

36. CHARGING IN VERSION 1

Version 1 should support **charging visibility and basic management**.

The operator can see:

- which drones are charging,
- battery level,
- charging station,
- estimated completion,
- availability.

But advanced autonomous charging decisions belong primarily to Version 2.

37. ALERT CENTER

Alerts should be categorized.

Information

Normal operational event.

Attention

Operator should review.

Warning

Potential operational problem.

Critical

Immediate action required.

Examples:

Attention

Drone 08 has lower-than-expected battery consumption efficiency.

Warning

Wind conditions increasing along current route.

Critical

Drone 12 has lost primary communications link.

38. INCIDENT MANAGEMENT

Every major incident should become an incident object.

Example:

INCIDENT #INC-104

TYPE
Communication Loss

ASSET
YS-DRONE-012

TIME
14:31

SEVERITY
CRITICAL

STATUS
ACTIVE

AUTOMATED RESPONSE
RETURN-TO-HOME INITIATED

OPERATOR
Joshua

ACTION
MONITORING

The incident should preserve a timeline.

39. ACTIVITY FEED

Mission Control should have a global operational activity stream.

Example:

14:32:11
Drone 014 entered Corridor B.

14:32:08
Route recalculated due to wind.

14:31:51
Delivery #10492 updated.

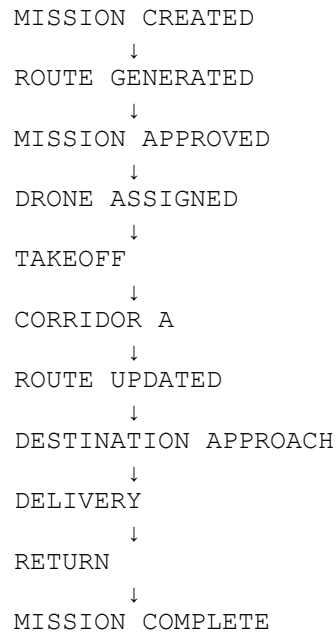
14:31:30
Drone 008 started charging.

14:31:02
AI detected vehicle near landing zone.

This becomes the operational history of the network.

40. MISSION TIMELINE

Every mission should have a timeline.



This will be extremely valuable for debugging and post-mission analysis.

41. MISSION REPLAY

Version 1 should ideally support historical mission replay.

An operator can select a completed mission and watch:

- route,
- drone position,
- altitude,
- telemetry,
- alerts,
- detected objects,
- route changes,
- mission events.

This is valuable for:

- operational review,
- engineering,
- incident investigation,
- customer support,
- investor demonstrations.

42. ROLE-BASED ACCESS

Mission Control should not give every employee the same authority.

Recommended roles:

1. Super Administrator

Full system access.

2. Operations Director

Network-level visibility and operational control.

3. Mission Operator

Can manage active missions.

4. Fleet Manager

Manages drones and fleet status.

5. Logistics Manager

Manages deliveries and logistics operations.

6. Maintenance Manager

Manages health and maintenance.

7. Autonomy Engineer

Access to autonomy diagnostics and engineering data.

8. Analyst

Read-only operational and historical analytics.

9. Viewer

Read-only dashboard.

43. PERMISSION MODEL

Permissions should be granular.

Example:

Action	Operator	Fleet Manager	Engineer	Admin
View fleet	✓	✓	✓	✓
Create mission	✓	✓	✓	✓
Approve mission	✓	✓	—	✓
Reroute	✓	✓	✓	✓
Emergency stop	✓	✓	✓	✓
Modify drone config	—	✓	✓	✓
Firmware update	—	—	✓	✓
Manage users	—	—	—	✓

44. AUTONOMY LEVELS

This is an important part of ySpaceOS.

The system should explicitly represent autonomy level.

Level 0 — Manual

Human controls the operation.

Level 1 — Assisted

System recommends actions.

Level 2 — Supervised Autonomy

System executes approved autonomous missions.

Level 3 — Conditional Autonomy

System can handle defined exceptions.

Level 4 — High Autonomy

System manages most operational decisions.

Level 5 — Network Autonomy

The logistics network coordinates itself with human oversight.

Version 1 should primarily operate around:

Level 1–2

Version 2 moves toward:

Level 2–4

This creates a clear long-term ySpaceOS roadmap.

45. AUTONOMY VISUALIZATION

The interface should show the autonomy state.

Example:

AUTONOMY

- SUPERVISED

Route: Autonomous
Obstacle Response: Autonomous
Mission Approval: Human
Emergency Override: Human

This makes the system's behavior transparent.

46. HUMAN OVERRIDE

No matter how autonomous the system becomes, Mission Control must provide an operator override mechanism.

Potential commands:

- Pause
- Hold
- Return
- Reroute
- Abort
- Land
- Emergency procedure

Critical commands should require confirmation where appropriate.

47. VERSION 1 AI ASSISTANT

A basic AI operational assistant can be included.

Example:

Operator: "Which drones are available for a 12 km delivery?"

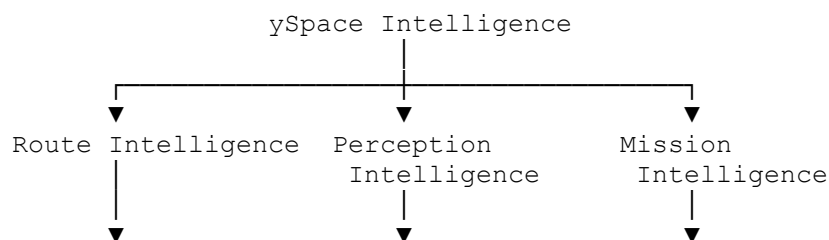
System:

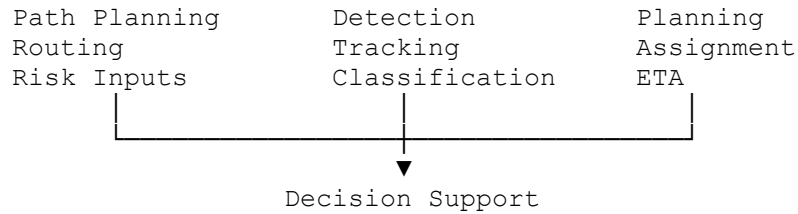
"Drones 04, 08 and 11 meet the current range and payload requirements. Drone 08 has the lowest predicted energy consumption."

This should initially be an **interface to the operational data**, not an unrestricted AI controlling aircraft.

48. VERSION 1 AI ENGINE ARCHITECTURE

The first version should contain several intelligence engines.





49. ROUTE INTELLIGENCE ENGINE

Inputs:

- origin,
- destination,
- drone,
- battery,
- payload,
- terrain,
- map,
- obstacles,
- weather,
- geofences,
- airspace,
- corridors.

Outputs:

- recommended route,
- alternative route,
- distance,
- estimated duration,
- estimated energy,
- risk score.

50. MISSION INTELLIGENCE ENGINE

Inputs:

- delivery request,
- fleet availability,
- drone capabilities,
- battery,
- location,

- route,
- priority.

Outputs:

- recommended drone,
- mission plan,
- estimated completion,
- recommended route.

51. PERCEPTION ENGINE

Inputs:

- camera,
- LiDAR,
- radar,
- other sensors,
- onboard perception outputs.

Outputs:

- object detection,
- object classification,
- object location,
- object tracking,
- confidence.

52. TELEMETRY ENGINE

Inputs:

- drone telemetry.

Outputs:

- current state,
- historical state,
- anomalies,
- operational metrics.

53. SPATIAL INTELLIGENCE ENGINE

Inputs:

- geospatial datasets,
- maps,
- 3D models,
- drone observations,
- infrastructure data.

Outputs:

- operational map,
- corridors,
- zones,
- route constraints,
- spatial context.

54. VERSION 2 — PRODUCT OBJECTIVE

Version 2 should answer a bigger question:

Can ySpaceOS predict operational problems and increasingly make logistics decisions itself?

This is where the platform moves from:

intelligent control tower

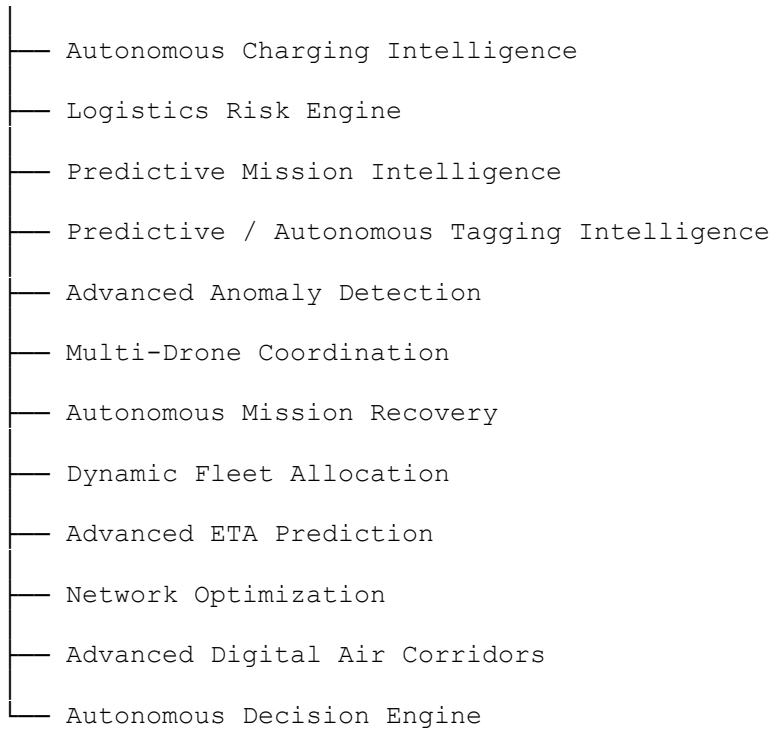
toward:

autonomous logistics operating system.

55. VERSION 2 FEATURE HIERARCHY

VERSION 2

├ Predictive Fleet Health



56. PREDICTIVE FLEET HEALTH OS

Instead of waiting for a drone to fail:

ySpaceOS predicts when something is becoming unhealthy.

The system analyzes:

- battery cycles,
- battery temperature,
- motor behavior,
- vibration,
- sensor performance,
- flight hours,
- historical failures.

Output:

Drone 014 battery health declining faster than fleet average.

Or:

Motor 02 shows abnormal vibration pattern. Maintenance recommended within 12 flight hours.

Predictive maintenance is therefore an intelligence layer rather than merely a maintenance checklist.

57. AUTONOMOUS CHARGING INTELLIGENCE

Version 2 introduces autonomous charging decisions.

The system considers:

- battery,
- mission queue,
- distance,
- priority,
- charger availability,
- expected demand,
- drone health,
- predicted future missions.

Example:

```
DRONE A
Battery: 22%
Next mission: 8 km
Priority: HIGH
```

```
DRONE B
Battery: 41%
Next mission: 15 km
Priority: LOW
```

```
CHARGER 01 AVAILABLE
```

The system may determine:

Charge Drone A first.

58. LOGISTICS RISK ENGINE

This becomes one of the most important Version 2 components.

It should predict:

- delivery delay,
- battery risk,
- weather risk,
- route risk,
- landing risk,
- network failure,
- asset failure,
- congestion,
- operational bottlenecks.

Instead of:

"Something went wrong."

the system should say:

"Mission 10492 has a 72% probability of delay within the next 8 minutes."

Then:

Recommended action: reroute through Corridor B.

59. PREDICTIVE MISSION INTELLIGENCE

The system should eventually predict mission outcomes before launch.

Example:

MISSION RISK

Completion Probability: 97%

On-Time Probability: 91%

Energy Risk: Low

Weather Risk: Medium

Landing Risk: Low

Communication Risk: Low

The operator can approve the mission with a much better understanding of its expected outcome.

60. PREDICTIVE / AUTONOMOUS TAGGING INTELLIGENCE

This can refer to the system automatically tagging operational entities based on observed behavior.

Examples:

"Potential landing obstruction"

"Frequently congested delivery zone"

"High-wind route segment"

"High-risk corridor"

"Reliable landing zone"

"Recurring delay location"

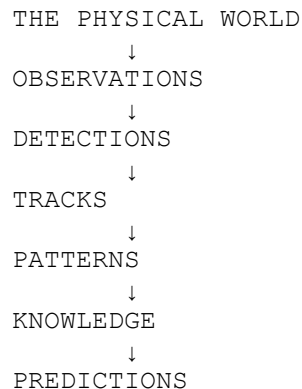
"Frequently detected obstacle"

The system is effectively creating an evolving operational knowledge layer over the physical world.

61. THE DIGITAL WORLD MODEL

This is one of the long-term strategic assets of ySpaceOS.

Over time, the system learns:



↓
DECISIONS

This is much more valuable than simply storing GPS tracks.

62. DIGITAL AIR CORRIDORS 2.0

Version 1:

Define and visualize safe operational corridors.

Version 2:

Continuously optimize those corridors based on actual operational data.

The system could learn:

- which corridors are faster,
- which consume more energy,
- which experience more wind,
- which have more obstacles,
- which have more communication problems,
- which are more reliable.

The corridor becomes an evolving intelligence object.

63. MULTI-DRONE COORDINATION

Version 2 should allow Mission Control to reason about multiple drones simultaneously.

For example:

5 DRONES
↓
SAME AREA
↓
OVERLAPPING ROUTES
↓
CONFLICT DETECTED
↓
ROUTES OPTIMIZED

The goal is not merely tracking five drones.

It is **coordinating a fleet**.

64. AUTONOMOUS MISSION RECOVERY

Suppose a drone encounters:

- weather,
- obstacle,
- low battery,
- lost communication,
- blocked landing zone.

Version 1 can alert the operator.

Version 2 can potentially:

1. detect the issue,
2. classify it,
3. calculate alternatives,
4. select an appropriate response,
5. execute the response within defined safety boundaries,
6. notify the operator.

That is the progression toward autonomy.

65. AUTONOMOUS DECISION ENGINE

The long-term architecture should include a decision layer.

```
OBSERVE
↓
UNDERSTAND
↓
PREDICT
↓
PLAN
↓
DECIDE
↓
ACT
```

↓
VERIFY

This is the core autonomous loop.

66. DATA ARCHITECTURE

Mission Control should operate on multiple classes of data.

Asset Data

Drone identity and capabilities.

Telemetry Data

Real-time operational state.

Mission Data

What the drone is supposed to do.

Spatial Data

Where everything exists.

Perception Data

What sensors observe.

Logistics Data

Deliveries and jobs.

Environmental Data

Weather and airspace.

Historical Data

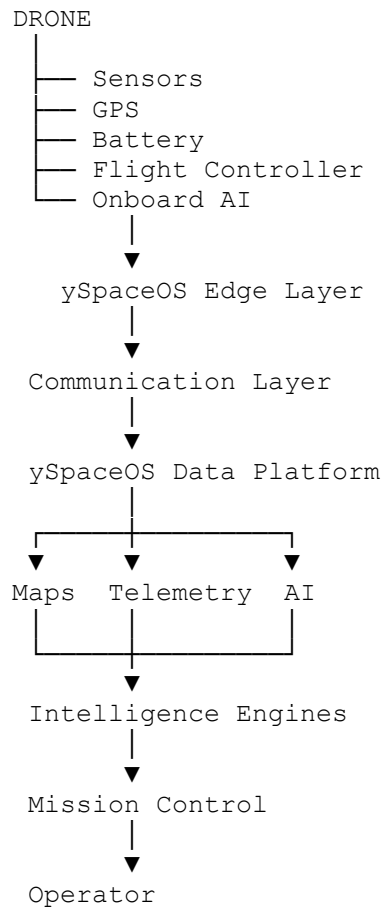
What happened previously.

Intelligence Data

Predictions and recommendations.

67. DATA FLOW

The high-level data flow:



68. EDGE VS CLOUD

This distinction should be explicitly maintained.

Onboard / Edge

Responsible for:

- immediate perception,
- low-latency obstacle avoidance,

- flight stabilization,
- immediate safety responses,
- local navigation.

ySpaceOS

Responsible for:

- mission planning,
- fleet coordination,
- route intelligence,
- spatial intelligence,
- historical analytics,
- prediction,
- logistics intelligence.

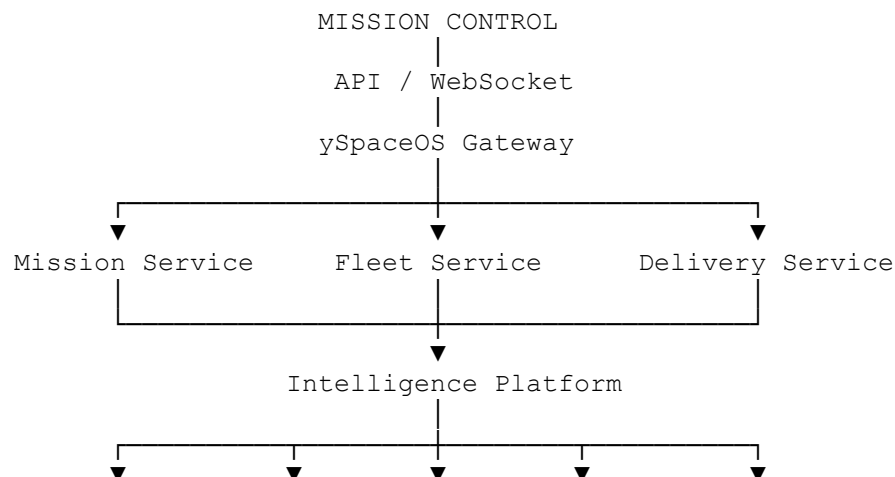
Mission Control

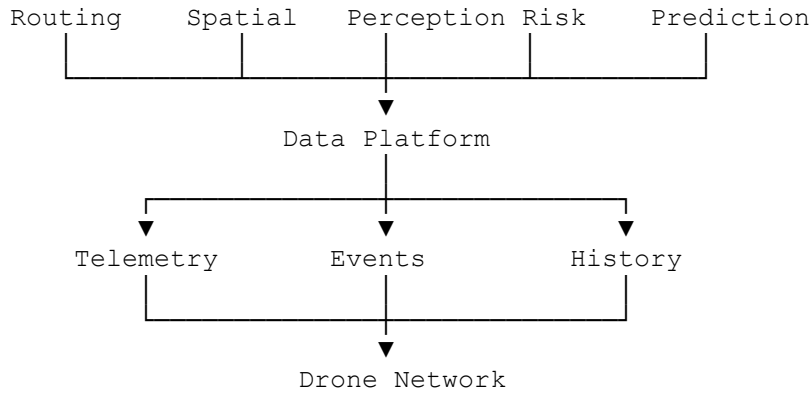
Responsible for:

- human interaction,
- visualization,
- command,
- monitoring,
- operational decisions.

69. SYSTEM ARCHITECTURE

Recommended conceptual architecture:





70. REAL-TIME COMMUNICATION

Mission Control needs real-time updates.

The operator should not have to refresh the page to see:

- drone movement,
- telemetry,
- mission updates,
- alerts,
- route changes.

The interface should receive events continuously.

71. EVENT-DRIVEN MODEL

Examples:

DRONE_POSITION_UPDATED

BATTERY_LEVEL_CHANGED

MISSION_STARTED

MISSION_COMPLETED

ROUTE_CHANGED

OBSTACLE_DETECTED

WEATHER_WARNING

DRONE_OFFLINE

DELIVERY_DELAYED

AI_OBJECT_DETECTED

CHARGING_STARTED

These events become the foundation of the operational timeline.

72. UI SCREEN INVENTORY

The designer should design at minimum:

Core

1. Login
2. Command Center
3. Live Operations
4. Fleet Overview
5. Drone Detail
6. Mission Overview
7. Mission Creation
8. Mission Detail
9. Mission Planning
10. Route Review

Intelligence

11. Route Intelligence
12. Spatial Intelligence
13. AI Detection
14. Airspace
15. Weather

Logistics

16. Deliveries
17. Delivery Detail
18. Bases
19. Charging

Operations

- 20. Alerts
- 21. Incidents
- 22. Activity Feed
- 23. Mission History
- 24. Mission Replay

Analytics

- 25. Fleet Analytics
- 26. Mission Analytics
- 27. Delivery Analytics
- 28. Route Analytics

Administration

- 29. Users
- 30. Roles
- 31. Permissions
- 32. System Settings

Version 2 then extends these screens rather than redesigning the entire product.

73. COMMAND CENTER — DESKTOP FIRST

Because this system will operate from Yspace headquarters, the primary design target should be a **large desktop control-room display**.

The UI should support:

- 1440p,
- 4K,
- large monitors,
- multiple monitors,
- widescreen control-room displays.

The designer should therefore think in terms of:

Operational workstation

rather than:

Mobile dashboard.

A responsive web interface can still exist, but the primary experience is the headquarters control tower.

74. MULTI-MONITOR FUTURE

Eventually Mission Control should support:

Monitor 1

Main operational map.

Monitor 2

Fleet + telemetry.

Monitor 3

Deliveries + missions.

Monitor 4

Alerts + incidents.

The product architecture should make this possible without requiring separate applications.

75. COMMAND CENTER LAYOUT

Recommended initial structure:

ySpaceOS MISSION CONTROL SYSTEM STATUS OPERATOR			
NAVIGATION	LIVE MAP		CONTEXT PANEL
Overview	DRONES / ROUTES /		Drone
Fleet	DELIVERIES / AIRSPACE		Mission
Missions			Alert
Deliveries			
Routes			

Spatial AI Alerts Analytics		
ACTIVITY / MISSION TIMELINE / ALERTS		

76. INFORMATION PRIORITY

The UI should prioritize:

Priority 1

Critical safety / operational information.

Priority 2

Active missions.

Priority 3

Fleet state.

Priority 4

Deliveries.

Priority 5

Environmental conditions.

Priority 6

Analytics.

Priority 7

Historical information.

77. DESIGN LANGUAGE

The visual language should communicate:

- precision,
- intelligence,
- technology,
- operational reliability,
- aviation,
- logistics,
- autonomy.

It should feel sophisticated without becoming visually noisy.

Avoid:

- excessive gradients,
- unnecessary animations,
- decorative charts,
- meaningless glowing elements,
- excessive cards,
- overly futuristic sci-fi interfaces.

This is a **real operations system**, not a movie interface.

78. DARK CONTROL-ROOM MODE

The primary interface should support a dark control-room-oriented theme.

The reason is practical:

- large-screen visibility,
- reduced visual distraction,
- map contrast,
- alert hierarchy,
- long-duration operations.

A light theme can exist later if useful, but the primary operational mode should be optimized for a control center.

79. COLOR SEMANTICS

Colors should communicate state consistently.

For example:

Green: normal

Yellow: attention

Orange: warning

Red: critical

Blue: information / selected state

Grey: inactive / unavailable

The system should not use color as the only indicator; icons, labels, and status text should also communicate meaning.

80. MAP LAYERS

The map should have a layer controller.

MAP LAYERS

- ☒ Drones
- ☒ Routes
- ☒ Deliveries
- ☒ Bases
- ☒ Charging
- ☐ Weather
- ☐ Wind
- ☐ Airspace
- ☐ Geofences
- ☐ Buildings
- ☐ Terrain
- ☐ AI Objects
- ☐ Traffic
- ☐ Digital Corridors

81. SEARCH

Mission Control should have global search.

Search:

- drone ID,
- mission ID,
- delivery ID,
- location,
- incident,
- customer,
- object ID.

Example:

Search "YS-014"

returns:

- drone,
- current mission,
- current location,
- health,
- historical missions.

82. GLOBAL COMMAND PALETTE

For expert operators, provide a command interface.

Example:

⌘ / CTRL + K

Then:

"Find Drone 014"

or:

"Show active delayed deliveries"

or:

"Open Mission 10492"

This can dramatically improve operational efficiency.

83. NOTIFICATIONS

Notifications should be grouped.

Operational

Mission updates.

Safety

Potential hazards.

Fleet

Drone health.

Logistics

Delivery issues.

System

Software/infrastructure issues.

84. NOTIFICATION DESIGN

Do not generate notifications for everything.

A notification should answer:

What happened?

Why does it matter?

What should I do?

Example:

Battery Warning

Drone 014 battery is below expected level for its current route.

Recommended action: Reroute to Base 02.

85. DESIGNING FOR COGNITIVE LOAD

This is one of the most important UX requirements.

Operators may eventually monitor dozens or hundreds of assets.

The interface must therefore avoid forcing humans to process every event individually.

The system should summarize.

Instead of:

38 telemetry events

show:

38 telemetry updates — all nominal

Instead of:

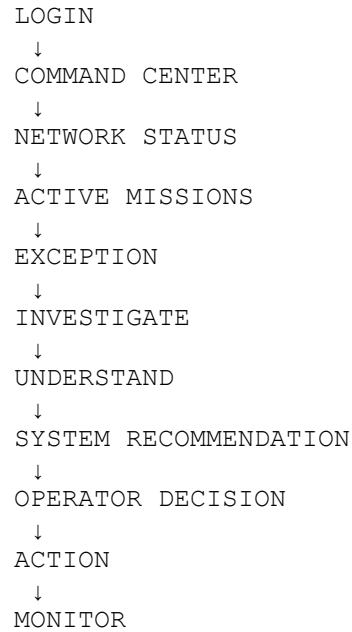
12 route events

show:

12 route changes — 11 routine, 1 requires attention

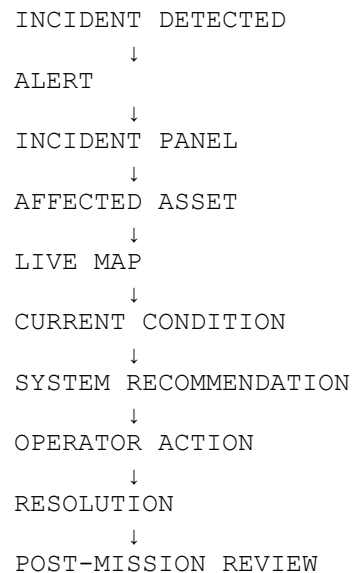
86. OPERATOR WORKFLOW

Typical workflow:



87. EMERGENCY WORKFLOW

If something serious happens:



88. MISSION APPROVAL UX

Before launch:

MISSION REVIEW

Destination
Victoria Island

Drone
YS-014

Distance
8.4 km

Estimated Flight
14 min

Battery at Launch
94%

Battery at Arrival
58%

Weather
Good

Route Risk
Low

Airspace
Clear

AUTONOMY
Supervised

[MODIFY] [APPROVE & LAUNCH]

This should be one of the most polished workflows in the product.

89. ROUTE REVIEW UX

When ySpaceOS generates a route, show:

Route A

Recommended.

Route B

Alternative.

Route C

Emergency / fallback.

For each:

- distance,
- duration,
- energy,
- risk,
- constraints.

Example:

ROUTE A
8.4 km
14 min
Low Risk
Recommended

ROUTE B
9.1 km
16 min
Very Low Risk

ROUTE C
7.9 km
13 min
Medium Risk

90. WHY THE SYSTEM CHOSE A ROUTE

This is essential.

Do not simply say:

AI Route Recommended

Instead:

Recommended because it minimizes estimated energy consumption while avoiding restricted airspace and current high-wind segments.

Explainability is essential when humans are supervising autonomous systems.

91. VERSION 1 MVP DEFINITION

The true Version 1 MVP should include:

Command

- Fleet monitoring
- Drone selection
- Basic commands
- Mission control

Missions

- Mission creation
- Mission planning
- Mission approval
- Mission execution
- Mission history

Routes

- Autonomous route generation
- Dynamic route updates
- Digital corridors
- Geofences

Telemetry

- Position
- Battery
- altitude
- speed
- connectivity
- mission state

Spatial

- 2D/3D map
- buildings
- zones
- routes
- assets

AI

- Object detection
- Object tracking
- Basic classification

Logistics

- Deliveries
- delivery status
- active jobs
- ETA

Operations

- Alerts
- incidents
- activity feed

Infrastructure

- bases
- charging visibility

92. VERSION 1 — WHAT NOT TO BUILD

This is equally important.

Do **not** attempt to build all of Version 2 into Version 1.

Avoid initially implementing:

- fully autonomous fleet management,
- predictive maintenance,
- autonomous charging optimization,
- advanced risk prediction,
- complete multi-drone autonomy,
- fully autonomous mission recovery,
- sophisticated logistics forecasting,
- autonomous network optimization.

The architecture should **support** them.

But the first product should not attempt to finish them.

93. VERSION 2 — EXPANSION

Version 2 adds:

Intelligence

- Predictive fleet health
- Logistics risk
- Mission prediction
- ETA prediction improvements
- anomaly detection

Autonomy

- autonomous charging,
- autonomous mission recovery,
- multi-drone coordination,
- fleet allocation,
- advanced route adaptation.

Spatial

- dynamic digital corridors,
- learned spatial tags,
- operational digital twin.

Operations

- predictive alerts,
- automated incident response,
- automated recommendations.

94. MVP VS FUTURE ROADMAP

Capability	V1	V2
Fleet Monitoring	✓	✓
Command & Control	✓	✓
Mission Planning	✓	Advanced
Dynamic Path Planning	✓	Advanced
Autonomous Route Intelligence	✓	Advanced

Capability	V1	V2
Digital Air Corridors	✓	Adaptive
Telemetry	✓	Predictive
Spatial Intelligence	✓	Advanced
AI Detection	✓	Advanced
AI Tracking	✓	Advanced
Delivery Operations	✓	Advanced
Alerts	✓	Predictive
Base Management	✓	Automated
Charging Visibility	✓	Autonomous Charging
Fleet Health	Basic	Predictive
Risk Engine	Basic rules	Predictive
Mission Recovery	Human-supervised	Autonomous
Multi-Drone Coordination	Basic	Advanced
Predictive Tagging	—	✓
Network Optimization	—	✓
Autonomous Decision Engine	Limited	Advanced

95. THE INVESTOR STORY

Version 1 should already demonstrate something important.

It should show that ySpace is not simply saying:

"We use drones for delivery."

It should demonstrate:

"We are building the software infrastructure that coordinates autonomous logistics."

That is a much stronger technology story.

The drone is the physical asset.

Mission Control is the operational interface.

ySpaceOS is the intelligence layer.

The long-term opportunity is the infrastructure connecting them.

96. WHY THE CONTROL TOWER MATTERS

The physical drone is only one component.

Imagine:

100 DRONES
+
1,000 DAILY DELIVERIES
+
MANY ROUTES
+
WEATHER
+
AIRSPACE
+
OBSTACLES
+
BATTERY CONSTRAINTS
+
CUSTOMER DEMAND

A human cannot manually optimize this entire system.

The control tower becomes necessary because the number of operational variables grows faster than human attention.

That is why Mission Control is not merely a dashboard.

It is the **operational brain interface**.

97. THE ySpaceOS DATA FLYWHEEL

Every mission creates data.

MISSION
↓

FLIGHT
↓
TELEMETRY
↓
OBSERVATION
↓
ROUTE DATA
↓
DELIVERY DATA
↓
OUTCOME
↓
LEARNING
↓
BETTER ROUTES
↓
BETTER PREDICTIONS
↓
BETTER OPERATIONS
↓
MORE DATA

This becomes an important long-term moat.

98. ROUTE INTELLIGENCE FLYWHEEL

Every completed route should contribute to future intelligence.

For example:

100 missions travel through a corridor.

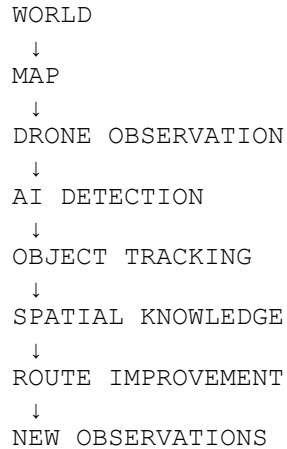
ySpaceOS learns:

- average flight time,
- energy consumption,
- wind behavior,
- obstacle frequency,
- communication reliability,
- landing reliability.

The next route recommendation becomes better because the system has historical evidence.

99. SPATIAL INTELLIGENCE FLYWHEEL

Likewise:



This means the environment itself becomes increasingly understood by ySpaceOS.

100. MISSION CONTROL AS A DIGITAL TWIN

Long-term, Mission Control should become a **digital representation of the ySpace logistics network**.

The operator should be able to see:

- physical assets,
- digital assets,
- missions,
- deliveries,
- infrastructure,
- routes,
- airspace,
- environmental conditions,
- detected objects,
- operational state.

The screen becomes a live digital twin of the logistics operation.

101. THE THREE-LAYER MODEL

The entire ySpaceOS architecture can ultimately be understood as three layers.

Layer 1 — Physical

Drones
Vehicles
Batteries
Sensors
Bases
Packages
Infrastructure

Layer 2 — Intelligence

Routing
Perception
Prediction
Risk
Spatial Intelligence
Mission Intelligence
Fleet Intelligence

Layer 3 — Control

Mission Control
Operators
Commands
Approvals
Monitoring
Decision Support

This is the conceptual foundation of the entire PRD.

102. DESIGN REQUIREMENT: EVERYTHING MUST HAVE CONTEXT

When the operator sees:

Battery: 38%

that number means little by itself.

Instead:

Battery: 38%

Estimated remaining range: 9.2 km

Current mission requires: 6.1 km

Status: Sufficient

That is useful information.

Mission Control should therefore prioritize **context over raw numbers**.

103. DESIGN REQUIREMENT: EVERYTHING IMPORTANT SHOULD BE ACTIONABLE

If the system shows:

High wind detected

the operator should be able to:

- inspect,
- view affected drones,
- see affected routes,
- review recommendation,
- approve rerouting.

Information should naturally connect to action.

104. DESIGN REQUIREMENT: EVERYTHING IMPORTANT SHOULD BE TRACEABLE

If a route changes:

The system should record:

WHO
WHAT
WHEN
WHY
FROM
TO

Example:

Route changed automatically at 14:32 because wind exceeded configured threshold.

This is essential for operational accountability.

105. AUDIT LOG

Every significant action should be recorded.

Examples:

- user logged in,
- mission created,
- mission approved,
- route changed,
- command issued,
- drone paused,
- alert acknowledged,
- incident resolved,
- configuration changed.

This creates an operational audit trail.

106. SECURITY

Mission Control controls physical assets.

Therefore, security is not optional.

Requirements should include:

- authentication,
- role-based authorization,
- secure communications,
- command authorization,
- audit logging,
- session management,
- device management,
- API authentication,
- encryption,
- monitoring.

The exact implementation should be finalized with engineering and security teams.

107. FAILSAFE DESIGN

Mission Control should never become a single point of failure for aircraft safety.

If the Mission Control interface disappears:

the drone must still follow its onboard safety logic.

This is another reason to separate:

Mission Control

from

onboard autonomy.

The FAA's UTM model similarly treats operator systems and distributed digital services as components that coordinate operations rather than replacing the aircraft's own operational safety mechanisms. ([Federal Aviation Administration](#))

108. OFFLINE / DEGRADED OPERATIONS

The interface should communicate system degradation.

Example:

NETWORK STATUS

- NORMAL
- DEGRADED
- LIMITED
- OFFLINE

The operator should know:

- what data is stale,
- which drones are affected,
- what commands remain available,
- when the last update occurred.

109. DATA FRESHNESS

Every real-time object should have a timestamp.

Example:

Updated 2 seconds ago.

If data becomes stale:

Last update: 31 seconds ago.

If critically stale:

Telemetry unavailable

This is extremely important in a control tower.

110. PERFORMANCE REQUIREMENTS

The UI should feel real-time.

The design team should target:

- fast initial loading,
- smooth map interaction,
- low-latency telemetry visualization,

- immediate alert rendering,
- efficient rendering of many map objects.

The exact engineering targets should be finalized after fleet scale is defined.

111. SCALABILITY

The system should not be designed only for:

5 drones.

The architecture should eventually support:

5 drones
↓
20
↓
50
↓
100
↓
500
↓
1,000+

The UI should remain understandable as fleet size grows.

This is why clustering, filtering, aggregation and exception-based design are important.

112. MAP CLUSTERING

If 500 drones occupy a dense region, do not render 500 giant markers.

Instead:

500
↓
Cluster
↓
120
↓
Zoom
↓
37

↓
Zoom
↓
Individual assets

113. FLEET OVERVIEW

The fleet screen should include:

TOTAL DRONES	32
IN MISSION	18
AVAILABLE	7
CHARGING	4
MAINTENANCE	2
OFFLINE	1

Then fleet health.

114. FLEET ANALYTICS

Metrics:

- missions completed,
- average flight time,
- utilization,
- distance,
- energy,
- delivery success,
- incidents,
- downtime,
- route efficiency.

Version 2 adds predictive analytics.

115. DELIVERY ANALYTICS

Metrics:

- total deliveries,
- on-time percentage,
- average delivery time,
- failed deliveries,
- delayed deliveries,
- average route distance,
- delivery density.

116. ROUTE ANALYTICS

Metrics:

- most-used corridors,
- fastest corridors,
- highest-energy routes,
- highest-risk routes,
- route deviation,
- average route efficiency.

117. OPERATIONAL SCORE

The dashboard can eventually provide a network health score.

Example:

YSPACE NETWORK HEALTH

92 / 100

Fleet	95
Missions	93
Routes	90
Deliveries	94
Airspace	91
Systems	97

This should be supplementary, not a replacement for real operational data.

118. COMMAND CENTER "NOW" VIEW

The main screen should answer:

What is happening right now?

It should display:

- active drones,
- active missions,
- active deliveries,
- critical alerts,
- route exceptions,
- airspace status,
- fleet health.

119. COMMAND CENTER "NEXT" VIEW

The system should also answer:

What is going to happen next?

Potential information:

- missions starting,
- drones becoming available,
- charging completion,
- predicted delays,
- upcoming weather,
- scheduled deliveries.

This becomes increasingly important in Version 2.

120. COMMAND CENTER "WHY" VIEW

When something happens:

Why?

The operator should be able to drill into the explanation.

Example:

Mission delayed

↓

Why?

↓

Wind increased by 21 km/h.

↓

Impact

Estimated energy consumption increased.

↓

Recommendation

Use Corridor B.

121. COMMAND CENTER "ACTION" VIEW

The system should always connect the insight to the available action.

Example:

RISK DETECTED

Mission #10492

Cause:
Wind

Impact:
+4 min ETA

Recommendation:

Reroute through Corridor B

[REVIEW ROUTE]
[APPROVE]
[DISMISS]

122. VERSION 1 SUCCESS METRICS

The first version should measure:

Operational

- mission completion rate,
- delivery completion rate,
- mission intervention rate,
- route success,
- incident response time.

Technical

- telemetry reliability,
- command latency,
- map rendering performance,
- system uptime.

Intelligence

- route recommendation acceptance,
- route efficiency,
- detection accuracy,
- tracking reliability.

123. VERSION 2 SUCCESS METRICS

Version 2 adds:

- reduction in manual interventions,
- reduction in delivery delays,
- reduction in emergency events,
- predictive maintenance accuracy,
- charging optimization,
- route efficiency improvements,

- autonomous recovery success.

124. PRODUCT NORTH STAR

The North Star Metric should ultimately reflect:

Percentage of logistics missions completed safely, successfully and efficiently with minimal human intervention.

This aligns the entire product toward autonomy without pretending autonomy is the goal for its own sake.

125. PHASED DEVELOPMENT

Phase 1A

Foundation.

- Authentication
- Command Center
- Fleet
- Map
- Telemetry
- Drone details

Phase 1B

Operations.

- Missions
- Deliveries
- Mission planning
- Route generation
- Alerts
- Activity

Phase 1C

Intelligence.

- Autonomous Route Intelligence
- Spatial Intelligence
- AI detection
- Tracking
- Digital Air Corridors

Phase 1D

Operational maturity.

- Analytics
- Mission replay
- Airspace
- Weather
- advanced permissions.

126. VERSION 2A

Predictive intelligence.

- Fleet health
- Risk engine
- predictive mission intelligence
- anomaly detection.

127. VERSION 2B

Autonomous operations.

- autonomous charging,
- mission recovery,
- multi-drone coordination,
- fleet allocation.

128. VERSION 2C

Network intelligence.

- adaptive corridors,
- predictive logistics,
- network optimization,
- operational digital twin.

129. FUTURE VERSION 3

Version 3 should eventually explore:

Network-level autonomous logistics.

At this point, ySpaceOS could potentially reason across:

- demand,
- fleet,
- routes,
- weather,
- infrastructure,
- charging,
- deliveries,
- airspace,
- maintenance.

Instead of asking:

"What should Drone 014 do?"

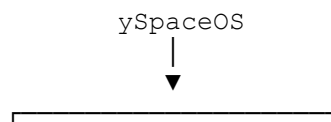
the system begins asking:

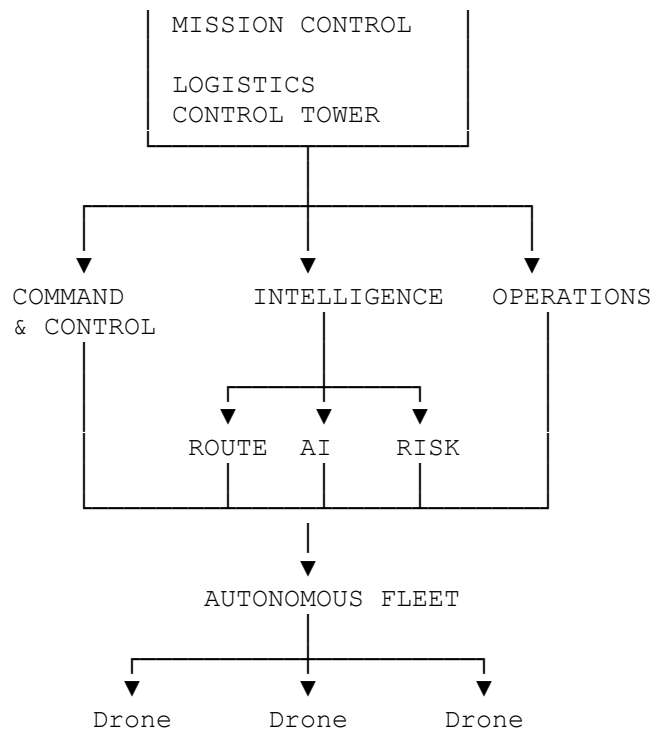
"What should the entire ySpace network do?"

That is the long-term vision.

130. THE FINAL PRODUCT MODEL

The product should ultimately be understood as:





131. THE MOST IMPORTANT PRODUCT DECISION

We should **not** design Version 1 as a collection of independent dashboards.

Everything should converge around one operational experience.

The operator should be able to move naturally:

Map → Drone → Mission → Route → Delivery → Alert → Intelligence → Action

without feeling like they have left the same system.

132. WHAT THE UI/UX DESIGNER SHOULD ACTUALLY DESIGN

The designer should not start by designing random screens.

The design process should begin with the **operational model**.

First:

Understand the fleet.

Second:

Understand missions.

Third:

Understand deliveries.

Fourth:

Understand spatial intelligence.

Fifth:

Understand route intelligence.

Sixth:

Understand alerts.

Seventh:

Understand operator decisions.

Then screens are designed around those workflows.

133. DESIGN DELIVERABLES

For Version 1, the designer should produce:

Information architecture

Complete navigation hierarchy.

User flows

Every major operational workflow.

Wireframes

All major screens.

High-fidelity designs

Final Mission Control interface.

Interactive prototype

Clickable operational flows.

Design system

Components, typography, spacing, icons, states.

Map system

Markers, routes, zones, layers, 3D objects.

Data visualization system

Charts, telemetry, analytics.

Alert system

Normal → attention → warning → critical.

Responsive behavior

Desktop, 4K, control-room displays.

134. REQUIRED DESIGN FLOWS

At minimum:

1. Login
2. View command center
3. Locate drone
4. Open drone
5. Create mission
6. Generate route

7. Review route
8. Approve mission
9. Launch mission
10. Monitor mission
11. Reroute mission
12. Pause mission
13. Return drone
14. Track delivery
15. Investigate alert
16. Handle incident
17. View AI detection
18. Inspect spatial object
19. View telemetry
20. View completed mission
21. Replay mission
22. View fleet
23. View base
24. View charging
25. View analytics

135. DESIGN COMPONENT SYSTEM

The design system should contain reusable:

- buttons,
- dropdowns,
- command buttons,
- status pills,
- alerts,
- map markers,
- route lines,
- telemetry cards,
- mission cards,
- drone cards,
- delivery cards,
- incident cards,
- charts,
- tables,
- timelines,
- modals,
- drawers,
- tooltips,
- map controls.

136. EMPTY STATES

Every screen must have meaningful empty states.

Example:

No active incidents

rather than a blank page.

137. ERROR STATES

The designer must design:

- drone offline,
- telemetry unavailable,
- map unavailable,
- route calculation failed,
- mission creation failed,
- command rejected,
- AI unavailable,
- stale data,
- communication degraded.

A control tower cannot be designed only for the happy path.

138. LOADING STATES

The system should communicate what is happening.

Example:

Calculating safest route...

rather than:

Loading...

This is especially important for AI-driven functionality.

139. AI CONFIDENCE

Where relevant, AI outputs should include confidence.

Example:

```
OBJECT DETECTED
```

```
Vehicle  
96% confidence
```

Or:

```
ROUTE RISK
```

```
Medium  
78% confidence
```

However, confidence must not be presented as a substitute for safety validation.

140. HUMAN TRUST

The UI must build trust in automation.

The system should never simply say:

AI decided.

Instead:

Recommended because...

This is one of the most important principles for the product.

141. AUTOMATION TRANSPARENCY

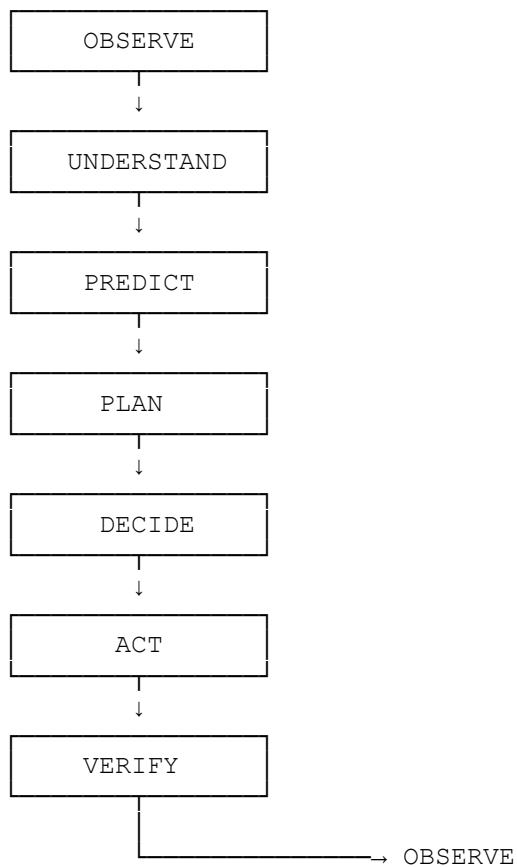
Every automated decision should ideally have:

DECISION
WHY
INPUTS
EXPECTED IMPACT
CONFIDENCE
AVAILABLE OVERRIDE

This becomes increasingly important as ySpace moves into Version 2.

142. THE ySpaceOS AUTONOMY LOOP

The long-term operating loop:



Version 1 implements this loop with significant human supervision.

Version 2 increasingly closes the loop automatically.

143. WHY VERSION 1 IS ALREADY INVESTOR-INTERESTING

Version 1 does **not** need predictive maintenance or fully autonomous fleet management to demonstrate technological differentiation.

A system that can:

- understand a 3D operational environment,
- generate autonomous routes,
- dynamically adjust routes,
- understand airspace,
- monitor telemetry,
- detect and track objects,
- manage missions,
- manage deliveries,
- coordinate drones,
- visualize the operation,
- and allow humans to supervise autonomy

is already much more than a conventional logistics dashboard.

That is the right V1 ambition.

144. WHY VERSION 2 MATTERS

Version 2 demonstrates the transition from:

software that helps operators

to:

software that increasingly performs operational reasoning.

The difference is:

Version 1

"Here is a problem."

Version 2

"Here is the problem, why it is happening, what will happen next, and what I recommend doing."

Future

"I detected the problem, evaluated the alternatives, executed the safest response within your operating policy, and notified you."

That is the trajectory of ySpaceOS.

145. THE FINAL PRODUCT POSITIONING

Internally, the product should be called:

ySpaceOS Mission Control

And its product description should be:

The autonomous logistics control tower for ySpaceOS.

Its purpose:

To give ySpace a unified command center for monitoring, planning, controlling, and increasingly automating autonomous transportation operations.

146. THE FINAL V1 DEFINITION

If we reduce Version 1 to one sentence:

ySpaceOS Mission Control V1 is a real-time autonomous logistics control tower that allows ySpace operators to monitor their fleet, manage deliveries, plan and supervise autonomous missions, dynamically optimize drone routes, understand the surrounding physical environment, receive telemetry, detect and track objects, and coordinate operations from a unified 2D/3D command interface.

147. THE FINAL V2 DEFINITION

Version 2:

ySpaceOS Mission Control V2 adds predictive and increasingly autonomous intelligence that enables the system to anticipate fleet, route, delivery and operational risks; optimize charging and fleet allocation; predict mission outcomes; automatically recover from defined failures; and increasingly make logistics decisions under human-defined operating policies.

148. THE LONG-TERM VISION

The ultimate goal is not simply:

"A dashboard for drones."

It is:

An operating system for autonomous logistics.

The physical drones are the machines.

The logistics network is the infrastructure.

The data is the nervous system.

The AI engines provide intelligence.

And Mission Control is the **control tower through which humans understand and supervise the entire system.**

149. PRODUCT DEVELOPMENT PRINCIPLE

One final rule should guide the entire build:

Build the architecture for the future, but build the product for the present.

We should architect ySpaceOS so that Version 2 and Version 3 are possible from the beginning.

But Version 1 should remain focused enough to actually ship.

That means:

V1

See → Understand → Plan → Control → Supervise

V2

Predict → Recommend → Optimize → Recover → Automate

Future

Coordinate → Decide → Act → Learn

150. FINAL MASTER ARCHITECTURE

